

Futcamedic API Reference

Complete REST API Documentation

Engineering Team
Futcamedic

May 5, 2026

Futcamedic API Reference

Base URL: <http://localhost:3000/api> (development) or <https://your-render-app.onrender.com/api> (production)

All endpoints require authentication via Supabase JWT in the Authorization header:

Authorization: Bearer <supabase_jwt_token>

Multi-tenancy is enforced via `school_id` from the authenticated user's profile.

Authentication Endpoints

POST /auth/register

Register a new school with admin user.

Request Body:

```
{
  "schoolName": "Academia de Fútbol XYZ",
  "fullName": "Juan Pérez",
  "email": "juan@xyz.com",
  "password": "SecurePass123!"
}
```

Response:

```
{
  "message": "School and admin created successfully",
  "user": { "id": "uuid", "email": "juan@xyz.com" },
  "school": { "id": "uuid", "name": "Academia de Fútbol XYZ" }
}
```

POST /auth/invite

Invite a user (teacher, admin, parent). Requires admin role.

Request Body:

```
{
  "email": "teacher@xyz.com",
  "fullName": "Carlos Gómez",
  "role": "profesor",
  "phone": "555-1234"
}
```

Response:

```
{
  "message": "User invited successfully",
}
```

```
"user": { "id": "uuid", "email": "teacher@xyz.com", "role": "profesor" }
}
```

Student Endpoints

GET /students

List students. Admin sees all; parent sees only their children.

Query Parameters: - category_id (optional): Filter by category - status (optional): Filter by status (activo, beca, pendiente, inactivo)

Response:

```
[
  {
    "id": "uuid",
    "full_name": "Juanito Pérez",
    "birth_date": "2012-05-15",
    "status": "activo",
    "uniform_delivered": false,
    "category": { "id": "uuid", "name": "U-12", "birth_year": 2012 },
    "parent": { "id": "uuid", "full_name": "María Pérez", "email": "maria@email.com" }
  }
]
```

POST /students

Create a new student. Requires admin role.

Request Body:

```
{
  "fullName": "Juanito Pérez",
  "birthDate": "2012-05-15",
  "categoryId": "uuid",
  "parentEmail": "maria@email.com",
  "parentName": "María Pérez",
  "parentPhone": "555-5678",
  "medicalNotes": "No allergies"
}
```

GET /students/:id

Get student details with attendance and payment history.

PUT /students/:id

Update student information.

Request Body:

```
{
  "fullName": "Juanito Pérez",
  "birthDate": "2012-05-15",
  "categoryId": "uuid",
  "parentId": "uuid",
  "medicalNotes": "Updated notes"
}
```

PATCH /students/:id/status

Change student status.

Request Body:

```
{
  "status": "beca"
}
```

Valid statuses: activo, beca, pendiente, inactivo

PATCH /students/:id/uniform

Toggle uniform delivery status.

Response:

```
{
  "message": "Uniform status updated",
  "uniform_delivered": true
}
```

DELETE /students/:id

Delete student (permanent). Creates audit record in deleted_students.

GET /students/deleted

List deleted students (admin only).

Attendance Endpoints**GET /attendances/category/:categoryId**

Get attendances for a category, filtered by date.

Query Parameters: - date (optional): Filter by date (YYYY-MM-DD)

GET /attendances/student/:studentId

Get attendance history for a student.

POST /attendances

Save bulk attendance (pass the list). Uses upsert to prevent duplicates.

Request Body:

```
{
  "trainingId": "uuid",
  "date": "2026-05-05",
  "type": "entrenamiento",
  "records": [
    { "studentId": "uuid", "present": true },
    { "studentId": "uuid", "present": false }
  ]
}
```

PATCH /attendances/trainings/:id/complete

Mark a training session as completed.

Response:

```
{
  "message": "Training marked as completed",
  "is_completed": true
}
```

Payment Endpoints

GET /payments

List payments.

Query Parameters: - type (optional): mensualidad or pago_profesor - month (optional): Filter by month (YYYY-MM format, e.g., "2026-05")

GET /payments/student/:studentId

Get payment history for a specific student.

GET /payments/pending

Get students with no payment in the current month.

POST /payments/students

Register payment for student(s). Supports single or bulk payments.

Request Body (Single):

```
{
  "studentId": "uuid",
  "amount": 500.00,
  "paymentDate": "2026-05-05",
  "description": "Mensualidad Mayo 2026"
}
```

Request Body (Bulk):

```
{
  "studentIds": ["uuid1", "uuid2", "uuid3"],
  "amount": 1500.00,
  "paymentDate": "2026-05-05",
  "description": "Mensualidad Mayo 2026 (Grupal)"
}
```

Note: For bulk payments, the amount is divided equally among students and recorded in payment_students.

POST /payments/teachers

Register payment to teacher (payroll).

Request Body:

```
{
  "teacherId": "uuid",
  "amount": 2000.00,
  "paymentDate": "2026-05-05",
  "description": "Pago quincenal Mayo"
}
```

Event Endpoints

GET /events

List events. Teachers see only events for their categories.

Query Parameters: - category_id (optional): Filter by category - month (optional): Filter by month (YYYY-MM)

GET /events/trainings

Get today's training sessions. Teachers see only their categories.

Query Parameters: - date (optional): Filter by date (YYYY-MM-DD), defaults to today

POST /events

Create a new event with optional recurrence.

Request Body (Single Event):

```
{
  "categoryId": "uuid",
  "venueId": "uuid",
  "date": "2026-05-10",
  "startTime": "16:00",
  "type": "entrenamiento",
  "description": "Entrenamiento técnico"
}
```

Request Body (Recurring Event - Weekly):

```
{
  "categoryId": "uuid",
  "venueId": "uuid",
  "date": "2026-05-10",
  "startTime": "16:00",
  "type": "entrenamiento",
  "description": "Entrenamiento semanal",
  "isRecurring": true,
  "recurringWeeks": 12,
  "recurrenceRule": {
    "frequency": "weekly",
    "days": ["Monday", "Wednesday", "Friday"]
  }
}
```

Request Body (Recurring Event - Biweekly):

```
{
  "categoryId": "uuid",
  "venueId": "uuid",
  "date": "2026-05-10",
  "startTime": "16:00",
  "type": "entrenamiento",
  "isRecurring": true,
  "recurringWeeks": 8,
  "recurrenceRule": {
    "frequency": "biweekly",
    "day": "Saturday"
  }
}
```

GET /events/:id/trainings

Get all training sessions generated from a recurring event.

POST /events/cancel

Cancel training session(s).

Request Body (Cancel single training):

```
{
  "trainingId": "uuid",
  "notifyParents": true
}
```

Request Body (Cancel entire event/recurrence):

```
{
  "eventId": "uuid",
  "cancelAll": true,
  "notifyParents": true
}
```

DELETE /events/:id

Delete an event and all its training sessions.

Category Endpoints

GET /categories

List all categories with assigned teacher.

GET /categories/mine

Get categories assigned to the current teacher.

POST /categories

Create a new category.

Request Body:

```
{
  "birthYear": 2012,
  "name": "U-12"
}
```

PATCH /categories/:id

Update category.

Request Body:

```
{
  "name": "U-12 Elite",
  "birthYear": 2012
}
```

POST /categories/:id/assign

Assign a teacher to a category.

Request Body:

```
{
  "teacherId": "uuid"
}
```

Venue Endpoints

GET /venues

List all venues.

POST /venues

Create a new venue.

Request Body:

```
{
  "name": "Campo Principal",
  "address": "Av. Reforma 123, Ciudad",
  "notes": "Césped artificial, vestidores disponibles"
}
```

PATCH /venues/:id

Update venue information.

DELETE /venues/:id

Delete a venue.

User Endpoints

GET /users

List users filtered by role.

Query Parameters: - role (optional): Filter by role (super_admin, admin, profesor, padre, alumno)

GET /users/teachers/:id

Get teacher details with assigned categories and email from auth.

PUT /users/:id

Update user profile.

Request Body:

```
{
  "fullName": "Carlos Gómez",
  "phone": "555-1234",
  "address": "Calle 123",
  "emergencyContactName": "Ana Gómez",
  "emergencyContactPhone": "555-5678",
  "medicalNotes": "Ninguna"
}
```

PATCH /users/me/password

Change current user's password.

Request Body:

```
{
  "currentPassword": "OldPass123!",
  "newPassword": "NewPass456!"
}
```

Teacher Permissions Endpoints

GET /permissions/mine

Get current teacher's permissions.

GET /permissions/:teacherId

Get permissions for a specific teacher (admin only).

PUT /permissions/:teacherId

Update teacher permissions (admin only).

Request Body:

```
{
  "can_manage_students": true,
  "can_manage_events": true,
  "can_view_finances": false,
  "can_manage_payments": false,
  "can_take_attendance": true,
  "can_manage_categories": false
}
```

School Endpoints

PUT /schools/:id

Update school information (admin only).

Request Body:

```
{
  "name": "Nueva Academia XYZ",
  "logoUrl": "https://supabase.storage.com/avatars/school-logo.png"
}
```

Error Responses

All endpoints return errors in this format:

```
{
  "error": "Error message describing what went wrong"
}
```

Common HTTP Status Codes: - 200 - Success - 201 - Created - 400 - Bad Request (validation error) - 401 - Unauthorized (invalid/missing token) - 403 - Forbidden (insufficient role/permissions) - 404 - Not Found - 409 - Conflict (duplicate record) - 500 - Internal Server Error

Middleware Stack

All API requests pass through this middleware chain:

1. **requireAuth** (api/middlewares/auth.middleware.ts)
 - Validates Supabase JWT from Authorization: Bearer <token> header
 - Sets req.user = { id, email, ...decodedJWT }
 2. **requireTenant** (api/middlewares/tenant.middleware.ts)
 - Looks up users table using req.user.id
 - Sets req.tenant = { school_id, role, user_id }
 - Rejects if user not found in users table
 3. **requireRole(...roles)** (applied per route)
 - Checks if req.tenant.role is in the allowed roles list
 - Rejects with 403 if role not authorized
-

Rate Limiting

Currently no rate limiting implemented. Consider adding for production: - Supabase Auth has built-in rate limiting for auth endpoints - Consider `express-rate-limit` for API endpoints in production